

美國住宅價格指數之時間數列分析與預測

114-2 時間數列分析期末報告

作者：114354025 統碩一 藍永濤

JUN. 2026

1. 摘要 (Abstract)

本文以美國住宅價格指數 (HPI) 作為分析對象，建立時間數列模型以描述其變動型態並預測短期走勢。資料分析顯示，HPI 具明顯長期上升趨勢，2002 年後成長加快，並在 2007 至 2010 年金融危機期間出現明顯下跌與較大波動。經 log 轉換與一階差分後，本文比較 ARIMA、SARIMA、ARIMAX 與 SARIMAX 等候選模型，並以 AICc、BIC、殘差診斷與樣本外預測誤差作為模型選擇依據。結果顯示，HPI 月成長率具有短期持續性、12 期相關結構，且金融危機 dummy 對成長率有顯著負向影響。綜合模型配適、預測表現與解釋性後，本文選擇 SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy 作為最終模型。預測結果顯示，該模型能捕捉 2012 年後 HPI 緩步回升趨勢，但對後段快速上升仍略有低估。

2. 緒論 (Introduction)

2.1 研究背景 (Background)

住宅價格是觀察房地產市場與經濟狀況的重要指標，住宅價格的變動會影響購屋者負擔、家庭財富、房貸市場與不動產投資決策。若住宅價格上升，可能代表市場需求增加；若住宅價格下跌，則可能反映景氣轉弱或市場信心下降。

因此，分析住宅價格的歷史變化與未來走勢具有實務意義。透過時間數列分析，可以觀察住宅價格是否具長期趨勢、季節性或其他規律，並提供購屋者、投資人、金融機構與政策制定者作為判斷房市變動的參考。

2.2 分析目標 (Objective of the Analysis)

本文以美國住宅價格指數 (HPI) 作為分析對象，目標是建立合適時間數列模型，描述其變動型態並預測未來短期走勢。研究中將比較 ARIMA、SARIMA、ARIMAX 與 SARIMAX 等候選模型，並根據模型配適、殘差診斷與預測表現選出最終模型。

3. 資料說明 (Data Description)

3.1 資料來源與變數說明 (Data Source)

本研究使用的資料為 [monthly-housing.csv](#)，取自 Texas Tech University 整理的 Time Series Data Sets 網站 Economics/Finance 類別，內容為美國住宅價格指數與既有住宅銷售數量月資料。

資料期間為 1991 年 1 月至 2013 年 4 月，資料頻率為月資料，共有 268 筆有效月觀察值。資料集中包含兩個主要變數：住宅價格指數 `hpi`，以及既有住宅銷售數量 `numsold`，其中 `numsold` 單位為千戶。

本文將以 HPI 作為主要分析變數。HPI 為住宅價格指數，用來衡量住宅市場整體房價相對於基準期的變化。

在本資料中，1991 年 1 月的 HPI 為 100，作為基準期，若某月 HPI 為 150，代表該月整體房價約為基準期的 1.5 倍；若 HPI 為 200，則代表約為基準期的 2 倍。HPI 並非實際房價金額，而是住宅價格相對變化的指標，透過建立單變量時間數列模型，探討美國住宅價格指數的時間變化與未來走勢。

3.2 資料清理：處理缺失值與日期格式整理 (Data Cleaning)

原始資料共有 280 筆觀測值與 3 個變數，其中 date 與 HPI 在末端有 12 筆缺失值。本研究主要分析對象為 HPI，且時間數列模型需完整且連續之時間索引，因此缺少日期或 HPI 觀測值將無法用於後續分析，故移除末端 date 與 HPI 為缺失值之資料。

接著將 date 變數轉換為日期格式並依照時間順序排列，整理後的資料期間為 1991 年 1 月至 2013 年 4 月，共 268 筆觀測值。

3.3 資料切分 (Train-Test Split)

經過資料清理後，本報告使用之 HPI 月資料期間為 1991 年 1 月至 2013 年 4 月，共 268 筆有效月觀察值。由於本報告的主要目標是建立時間數列模型，並評估其對未來住宅價格指數的短期預測能力，因此後續分析將採用時間順序切分資料，而非隨機切分。

將樣本期間最後連續 12 個月作為測試資料；其餘較早期間的資料則作為訓練資料，用於探索性資料分析、平穩性檢查、轉換與差分、ACF/PACF 分析、候選模型建立與參數估計。

具體而言，訓練資料期間為 1991 年 1 月至 2012 年 4 月，共 256 筆；測試資料期間為 2012 年 5 月至 2013 年 4 月，共 12 筆。後續凡涉及模型判斷與模型建立之分析，皆以訓練資料為基礎，以避免測試期間資訊影響模型選擇。最後再使用訓練資料所建立之模型預測測試期間的 HPI，並將預測結果與實際值比較，以 RMSE、MAE、MAPE 等指標評估模型的預測能力。

4. 探索性資料分析 (EDA)

本章使用切分後的 HPI 月資料訓練集進行探索性資料分析，主要目標是觀察住宅價格指數的長期趨勢、波動情形、可能的季節性與異常變化，作為後續模型識別與轉換處理的依據。

4.1 時間數列圖 (Time Series Plot)

由圖 4.1 可以觀察到，HPI 在 1991 年至 2013 年間並非圍繞固定水準上下波動，而是呈現明顯長期變化。HPI 從 1991 年約 100 開始緩慢上升，2000 年後上升速度加快，並在 2006 至 2007 年附近達到高點。之後 HPI 明顯下跌，約在 2011 年後才逐漸回升。這表示 HPI 的平均水準會隨時間改變，因此從圖形上初步判斷，HPI 具非平穩特徵。

由於本圖為長期月資料，單一時間序列圖不易清楚判斷短期季節性；由此圖只能看出並無非常明顯的規律季節波動，是否存在季節性仍需進一步透過月份分組圖或 ACF 圖確認。

此外，圖中沒有明顯單一月份的極端離群值。2000 年後，HPI 的上升速度較前期加快，並在 2006 至 2007 年附近達到高點；之後出現明顯下跌，顯示房價可能受到景氣循環或房市修正影響。至於變異程度是否隨時間改變，僅由原始 HPI 時間數列圖不易明確判斷。後續可透過 $\log(\text{HPI})$ 觀察相對變動是否較穩定，並透過差分後序列檢視去除長期趨勢後的月變化幅度。

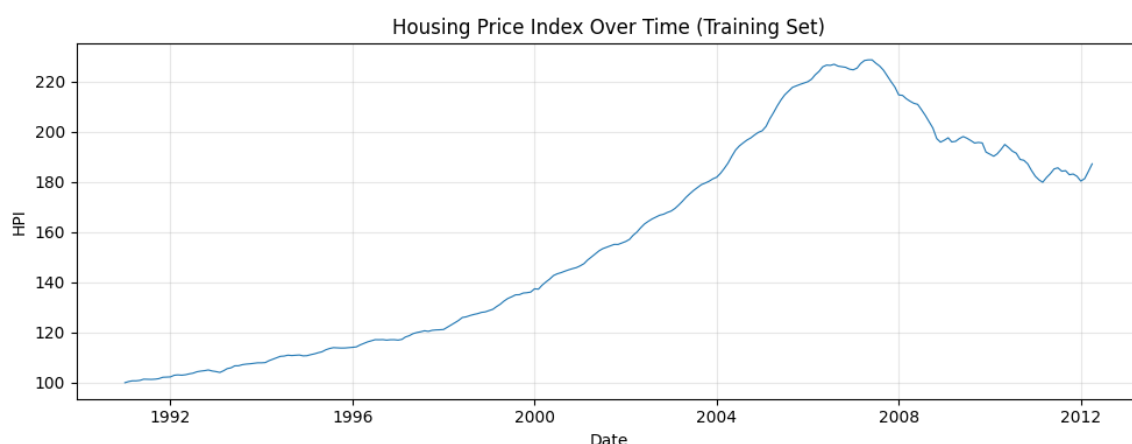


圖 4.1: HPI 時間數列圖

4.2 敘述統計量 (Summary Statistics)

計算 HPI 的敘述統計量，以了解資料在樣本期間的中心位置、分散程度與分布情形。HPI 的平均數為 157.199，中位數為 154.290，兩者相當接近，未顯示明顯偏態。標準差為 42.155，進一步以變異係數衡量，標準差占平均數的 26.8%，顯示 HPI 在樣本期間具有一定程度的分散與變動。

HPI 的最小值為 100.000，接近資料起點的基準水準。最大值為 228.710，與時間序列圖中 2006 至 2007 年附近的房價高點相符。第一四分位數為 116.117，第三四分位數為 195.122，四分位距約為 79.005，顯示樣本期間 HPI 的分布範圍不小。整體而言，敘述統計量與時間序列圖一致，顯示 HPI 在樣本期間存在明顯水準變化，後續需進一步檢查平穩性並考慮轉換或差分處理。

	Mean	SD.	Min.	Q1	Med.	Q3	Max.	CV.
HPI	157.199	42.155	100.000	116.117	154.290	195.122	228.710	0.268

表 4.1: HPI 敘述統計量

5. 模型識別 (Model Identification)

5.1 平穩性檢查 (Stationarity Inspection)

本節將透過 time series plot、12-month rolling mean、12-month rolling standard deviation、ACF plot 與 seasonal plot 與 ADF test，檢查 HPI 序列是否具平穩性。

由圖 5.1 可見，HPI 在 1991 年至 2007 年間明顯上升，之後下跌，並於樣本後期回升，顯示序列存在長期趨勢，平均水準非固定。

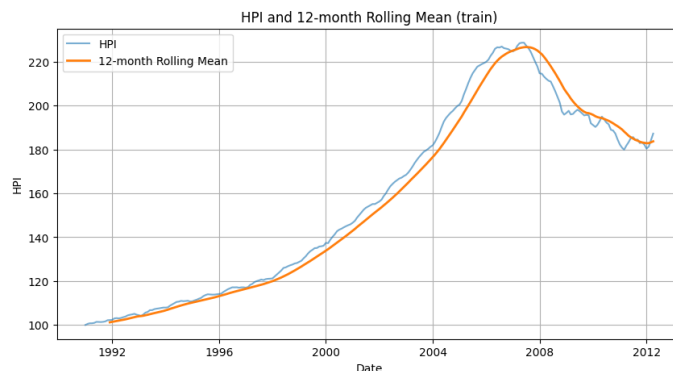


圖 5.1: 訓練集 12-month rolling mean

由圖 5.2，12-month rolling standard deviation 在不同期間有明顯高低變化，尤其 2004 年後波動幅度增加，顯示變異數也不穩定。

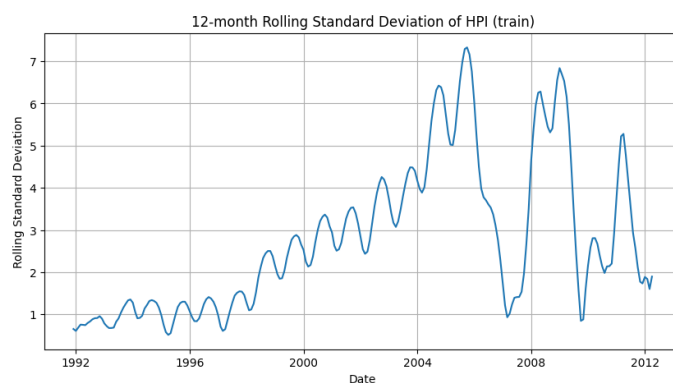


圖 5.2: 訓練集 12-month rolling standard deviation

由圖 5.3，在季節性方面，ACF plot 雖在 lag 12、24、36 仍呈現高度相關，但整體 ACF 是緩慢下降，主要反映趨勢與非平穩性，而非明顯季節性。seasonal plot 也未顯示固定月份重複出現的高低型態，因此 HPI 月度季節性不明顯。

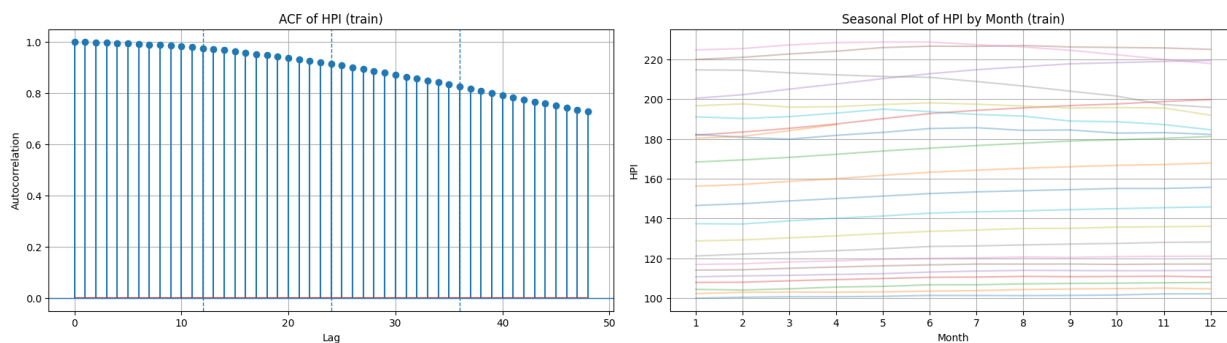


圖 5.3: 訓練集 ACF 與 seasonal plot

最後，由時間序列圖與 rolling mean 可觀察到，HPI 在樣本期間具明顯長期趨勢，因此本研究採用含截距與線性趨勢項之 ADF 檢定。檢定結果顯示 $p\text{-value} = 0.1638$ ，在 5% 顯著水準下未能拒絕序列具有單位根之虛無假設，表示原始 HPI 序列在統計上不具平穩性。

綜合圖形分析與檢定結果，判定原始 HPI 序列不具平穩性，後續模型建立前需考慮對數轉換與差分處理。

5.2 轉換與差分 (Transformation and Differencing)

根據前一節平穩性檢查，原始 HPI 數列具明顯長期趨勢，不適合直接用於 ARIMA 類模型。參考 Larson (2011) 在房價預測研究中使用 log house price index 與其變化量進行分析，並比較不同差分形式下的預測模型，本報告先對 HPI 取 log，以降低不同價格水準下變動幅度不一致的問題。由於 HPI 為正值價格指數，log 轉換也具有實務解釋上的合理性。

接著，為了移除原始序列中的長期趨勢，對 $\log(\text{HPI})$ 進行一階差分，差分後資料可近似解釋為 HPI 的月成長率。log 差分的主要目的不是消除所有資料特徵，而是將原本具有長期水準變化的 HPI 轉換為月變動率，使其更接近可建模的時間數列。

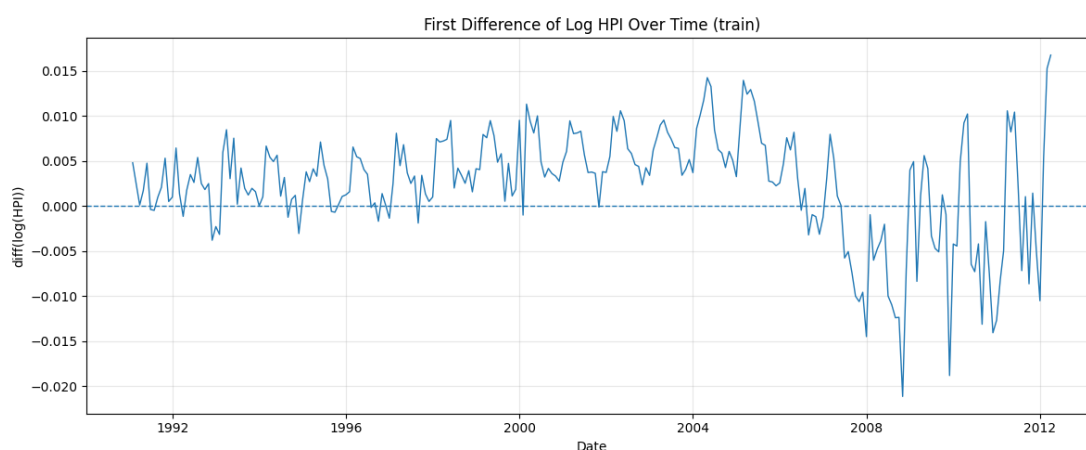


圖 5.4: $\text{diff}(\log(\text{HPI}))$ 時間數列圖

由差分後序列圖可觀察到， $\text{diff}(\log(\text{HPI}))$ 已不再呈現原始 HPI 中明顯的長期上升趨勢，而是大致圍繞 0 上下波動，表示一階差分已有效降低趨勢性。然而，log 差分並不會消除特殊經濟事件或房市循環造成的影響。約 2002 年後，HPI 月成長率多數時間維持在正值且呈上升趨勢，反映房價上升速度加快；2007 至 2010 年間，序列明顯轉為負值且波動幅度擴大，可能與金融危機及房市修正有關；2011 年後則逐漸回升。

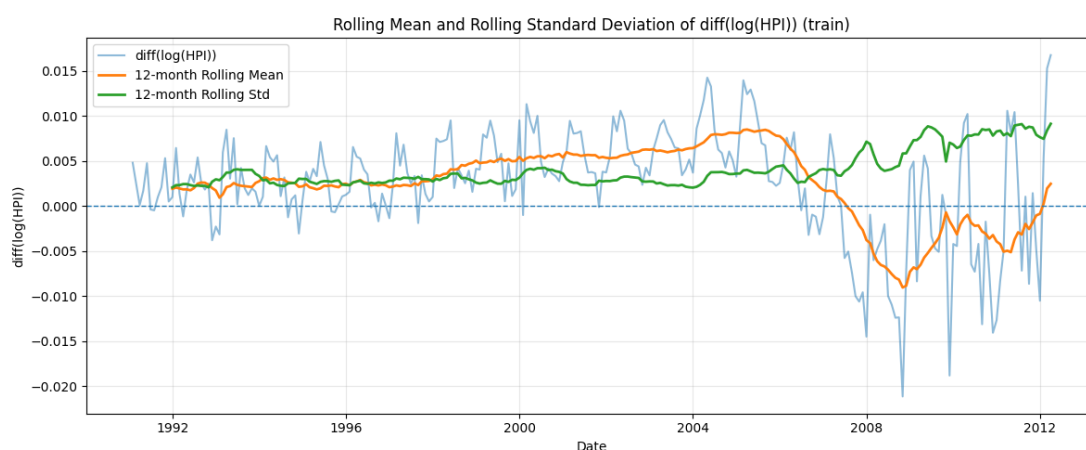


圖 5.5: $\text{diff}(\log(\text{HPI}))$ 之 rolling mean 與 rolling standard deviation

進一步觀察 12-month rolling mean，可發現差分後序列的平均水準雖比原始 HPI 穩定，但仍隨不同經濟階段改變。rolling standard deviation 在 2007 年後明顯增加，表示危機期間 HPI 月成長率波動較大。

此外，ADF 檢定結果顯示 $\text{diff}(\log(\text{HPI}))$ 的 p-value 為 0.1358，大於 0.05，因此仍無法拒絕單根虛無假設。這表示 log 差分雖然改善了原始 HPI 的長期趨勢問題，但不能完全消除特殊時期造成的非平穩性。後續將以 $\text{diff}(\log(\text{HPI}))$ 作為主要建模基礎，並透過 ACF、PACF、候選模型比較與殘差診斷，進一步選擇合適的 ARIMA 或 ARIMAX 模型。

5.3 ACF / PACF 圖

本節以訓練資料之 $\text{diff}(\log(\text{HPI}))$ 觀察 ACF 與 PACF，以作為後續 ARIMA 與 SARIMA 候選模型設定的依據。

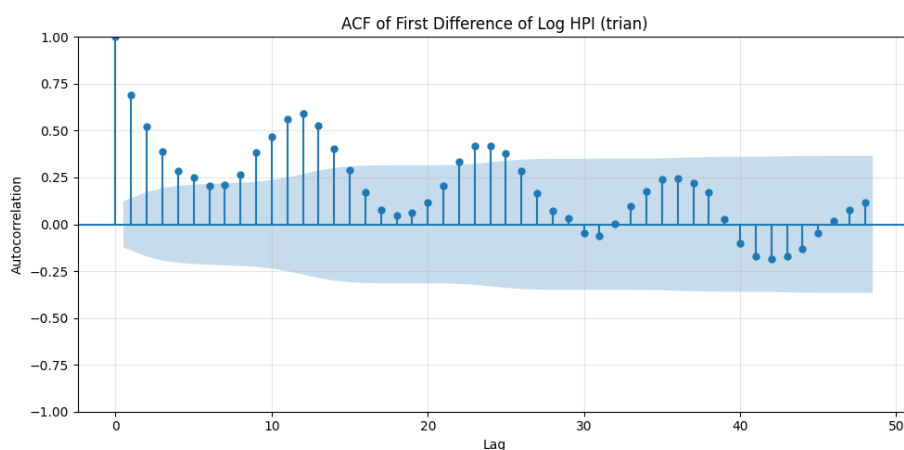


圖 5.6: $\text{diff}(\log(\text{HPI}))$ ACF 圖

由 ACF 圖可見，lag 1 至 lag 3 皆呈現明顯正相關，表示 HPI 月成長率具有短期持續性，並非白噪音序列。此外，ACF 在 lag 12 與 lag 24 附近仍有相對較高的正相關，顯示序列可能存在接近 12 個月的週期性或弱季節性相關。不過，此相關型態呈現較平滑的波動型態，不宜直接判定具有明確固定季節性。

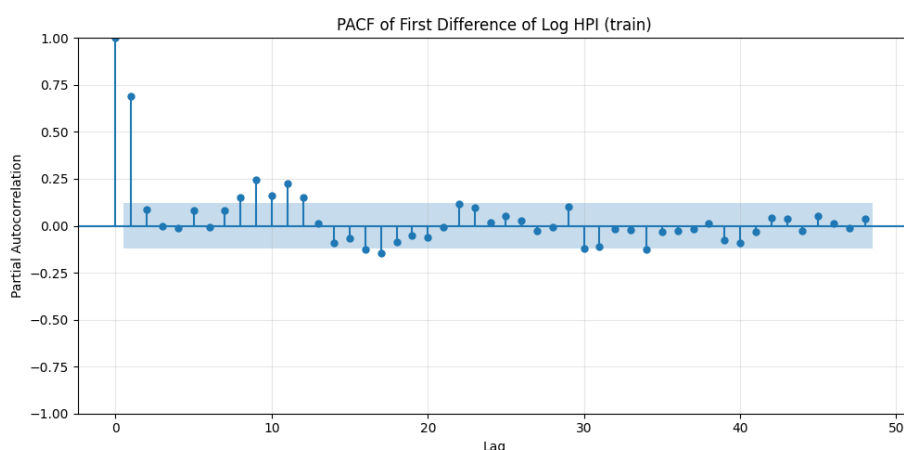


圖 5.7: $\text{diff}(\log(\text{HPI}))$ PACF 圖

由 PACF 圖可見，lag 1 的部分自我相關最為明顯，之後大多數 lag 落在信賴區間內，但 lag 9 至 lag 12 附近仍有部分相關性偏高。此結果顯示非季節部分可先考慮 AR(1) 結構，同時也可將 12 期季節項納入候選模型比較。

5.4 候選模型 (Candidate Models)

根據前述平穩性檢查與 ACF/PACF 圖形判斷，原始 HPI 序列具明顯長期趨勢，因此將以 $\log(\text{HPI})$ 作為正式建模序列，並在 ARIMA 類模型中設定一階差分，即 $\text{ARIMA}(p,1,q)$ 。前述 $\text{diff}(\log(\text{HPI}))$ 主要用於檢查平穩性與判斷 ACF/PACF 結構，而正式模型則以 $\log(\text{HPI})$ 進行估計，使預測結果可再轉換回原始 HPI 水準進行解釋。

首先考慮 ARIMA 與 SARIMA 模型。由於 HPI 長期而言具上升趨勢，若模型未加入 drift，將隱含差分後序列的平均值設為 0，亦即平均月成長率為 0，這與資料特徵不完全相符。因此，本研究在 ARIMA 與 SARIMA 模型中加入 drift term，用以捕捉整段訓練期間的固定平均月成長率。Random walk with drift 則作為 baseline model，用以檢查較複雜模型是否能提供更好的模型表現。

在非季節 ARIMA 模型方面，由 $\text{diff}(\log(\text{HPI}))$ 的 ACF 可觀察到前幾期仍有明顯正相關，而 PACF 在 lag 1 最為明顯，因此先考慮 $\text{ARIMA}(1,1,0)$ with drift。此外，為比較 AR 與 MA 結構對模型表現的影響，進一步納入 $\text{ARIMA}(1,1,1)$ with drift 與 $\text{ARIMA}(2,1,1)$ with drift 作為候選模型。

在季節模型方面，雖然 seasonal plot 未顯示明確固定月份型態，但 $\text{diff}(\log(\text{HPI}))$ 的 ACF 在 lag 12 與 lag 24 附近仍有相對較高的正相關，表示序列可能存在接近 12 個月的週期性或弱季節性相關。因此，本研究納入 $\text{SARIMA}(1,1,1)(1,0,0)[12]$ with drift 與 $\text{SARIMA}(1,1,1)(0,0,1)[12]$ with drift，以檢查加入 12 期季節 AR 或季節 MA 項是否能改善模型表現。

此外，EDA 顯示 HPI 在 2002 年後成長速度加快，並在 2007 至 2010 年間出現明顯下跌與較大波動，可能反映房市循環或總體經濟事件影響。因此，本研究亦考慮加入事件 dummy

variables 之 ARIMAX 與 SARIMAX 模型。這類模型不加入 drift，而是改由 dummy variables 捕捉不同經濟時期的平均月成長率變化，以避免 drift 與 dummy 同時解釋平均成長率。

綜合上述，候選模型包括：

1. Random walk with drift
2. ARIMA(1,1,0) with drift
3. ARIMA(1,1,1) with drift
4. ARIMA(2,1,1) with drift
5. ARIMAX(1,1,1) with crisis dummy
6. ARIMAX(1,1,1) with post-2002 dummy and crisis dummy
7. SARIMA(1,1,1)(1,0,0)[12] with drift
8. SARIMA(1,1,1)(0,0,1)[12] with drift
9. SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy
10. SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy

5.5 模型選擇準則 (Model Selection Criteria)

本報告將由模型配適、樣本外預測能力與殘差診斷三個面向選擇最終模型。首先，針對各候選模型在訓練資料上的配適結果，使用 AICc 與 BIC 作為模型選擇指標。AICc 用於衡量模型配適程度並修正有限樣本下的模型複雜度問題，比 AIC 更適合此次樣本；BIC 則對參數數量給予較高懲罰，用以檢查較簡潔的模型是否已足夠描述資料結構。

其次，為評估模型的樣本外預測能力，將以 RMSE、MAE 與 MAPE 衡量 2012 年 5 月至 2013 年 4 月的 HPI 預測表現，其中 RMSE 對較大的預測誤差較敏感，MAE 反映平均絕對誤差大小，MAPE 則以百分比形式呈現預測誤差，便於比較與實務解釋。

最後進行殘差診斷，以確認模型是否已充分捕捉資料中的時間相依結構。診斷內容包括殘差時間序列圖、殘差 ACF 圖與 Ljung-Box test。若殘差仍存在明顯自相關，表示模型可能尚未充分描述序列結構，即使其 AICc 或預測誤差較低，也不宜直接作為最終模型。

綜合而言，最終模型不會僅依據單一指標選擇。若不同指標指向不同模型，將優先選擇測試集預測表現穩定、殘差接近白噪音，且模型結構不過度複雜者，作為兼具統計合理性與實務解釋性的最終模型。

6. 模型估計 (Model Estimation)

本章針對前述候選模型進行參數估計。所有模型皆使用訓練資料期間 1991 年 1 月至 2012 年 4 月進行估計，並以 maximum likelihood estimation 估計模型參數。表 6.1 整理較具代表性模型的主要參數估計結果。

ARIMA(1,1,1) with drift 的 AR(1) 係數為正且顯著，表示 HPI 的月成長率具有明顯短期持續性。Drift 係數約為 0.0027，且達統計顯著，表示訓練期間 HPI 平均每月約有 0.27% 的正成長。此結果與 HPI 長期上升的資料特徵一致。

SARIMA(1,1,1)(1,0,0)[12] with drift 的 AR(1)、MA(1)、seasonal AR(1)與 drift 達統計顯著。其中 seasonal AR(1)係數為 0.3781，表示 HPI 月成長率不只受短期變動影響，也與約 12 個月前的變動有關。此結果與前述 ACF 圖中 lag 12 附近仍存在相關性的觀察一致。不過，由於 seasonal plot 並未顯示明確固定月份型態，因此此處較適合解釋為 12-month dependence，而非強烈季節性。

ARIMAX 模型的 crisis dummy 皆為負且顯著，表示 2007 - 2010 年金融危機期間，HPI 的平均月成長率明顯下降。以同時包含 post-2002 與 crisis dummy 的 ARIMAX 模型為例，crisis dummy 估計值為 -0.0076，表示 crisis 期間 HPI 的平均月成長率約下降 0.76 個百分點。相較之下，post-2002 dummy 雖為正，但未達統計顯著，表示在控制金融危機效果與 ARMA 誤差結構後，2002 年後平均成長率提高的證據較弱。

SARIMAX 模型同時考慮 12 期相關結構與事件 dummy variables。估計結果顯示，seasonal AR(1)係數仍為正且顯著，代表 12 期相關結構在加入 dummy variables 後仍然存在。此外，crisis dummy 仍為負且顯著，表示金融危機期間對 HPI 月成長率具有明顯負向影響。這說明 SARIMAX 模型能同時捕捉 HPI 的年度相關結構與金融危機期間的結構性下滑。

整體而言，估計結果顯示 HPI 月成長率具有短期持續性、12 期相關結構，且金融危機期間存在顯著負向影響。這些參數方向皆符合住宅價格資料的實際背景，因此模型設定在應用上具合理性。後續章節將進一步透過殘差診斷與樣本外預測表現，判斷各模型是否能充分捕捉資料中的時間相依結構。

Model	Parameter	Estimate	Std. Error	p-value	Interpretation
ARIMA(1,1,1) with drift	AR(1)	0.7687	0.0561	< 0.001	月成長率具有短期持續性
ARIMA(1,1,1) with drift	MA(1)	-0.1239	0.0839	0.139	短期 shock effect 不明顯
ARIMA(1,1,1) with drift	Drift	0.0027	0.001	0.008	平均月成長為正
SARIMA(1,1,1)(1,0,0)[12] with drift	AR(1)	0.7653	0.0729	< 0.001	月成長率具有短期持續性
SARIMA(1,1,1)(1,0,0)[12] with drift	MA(1)	-0.302	0.1189	0.011	短期 shock adjustment 顯著
SARIMA(1,1,1)(1,0,0)[12] with drift	Seasonal AR(1)	0.3781	0.0738	< 0.001	存在 12 期相關結構

SARIMA(1,1,1)(1,0,0)[12] with drift	Drift	0.0026	0.0012	0.023	平均月成長為正
ARIMAX(1,1,1) with crisis dummy	Crisis dummy	-0.0069	0.0015	< 0.001	危機期間月成長 率顯著下降
ARIMAX(1,1,1) with post- 2002 and crisis dummies	Post-2002 dummy	0.0011	0.0013	0.398	2002 後成長提高 的證據較弱
ARIMAX(1,1,1) with post- 2002 and crisis dummies	Crisis dummy	-0.0076	0.0017	< 0.001	危機期間月成長 率顯著下降
SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy	Seasonal AR(1)	0.3221	0.0751	< 0.001	存在 12 期相關 結構
SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy	Crisis dummy	-0.005	0.0018	0.004	危機期間月成長 率顯著下降
SARIMAX(1,1,1)(1,0,0)[12] with post-2002 and crisis dummies	Seasonal AR(1)	0.3202	0.0759	< 0.001	存在 12 期相關 結構
SARIMAX(1,1,1)(1,0,0)[12] with post-2002 and crisis dummies	Post-2002 dummy	0.0003	0.0015	0.822	2002 後成長提高 的證據不明顯

表 6.1: 模型參數估計結果

7. 模型診斷 (Model Diagnostics)

本章針對所有候選模型進行殘差診斷，以檢查模型是否充分捕捉資料中的時間相依結構。理想上，良好時間數模型應使模型殘差近似 white noise，且不應存在自相關。

本文使用殘差時間數列圖、殘差 ACF 圖、Ljung-Box test、直方圖與 Q-Q plot 檢查殘差性質，呈現於圖 7.1 與表 7.1。其中，Ljung-Box test 的虛無假設為殘差在指定 lag 內不存在自我相關，使用 lag 24 進行檢定，檢查模型是否殘留兩年內之自相關結構。

由於所有候選模型 Ljung-Box p-value 皆小於 0.05，因此沒有模型能在嚴格標準下完全通過 white-noise residual 檢定。因此，本研究不採用單純 pass/fail 機制，而是使用相對診斷表現作為模型保留依據。具體而言，優先保留 Ljung-Box statistic 較小，且 maximum absolute residual ACF 較低的模型。這些模型雖未完全消除殘差自相關，但相較於其他候選模型，殘差行為較接近 white noise。

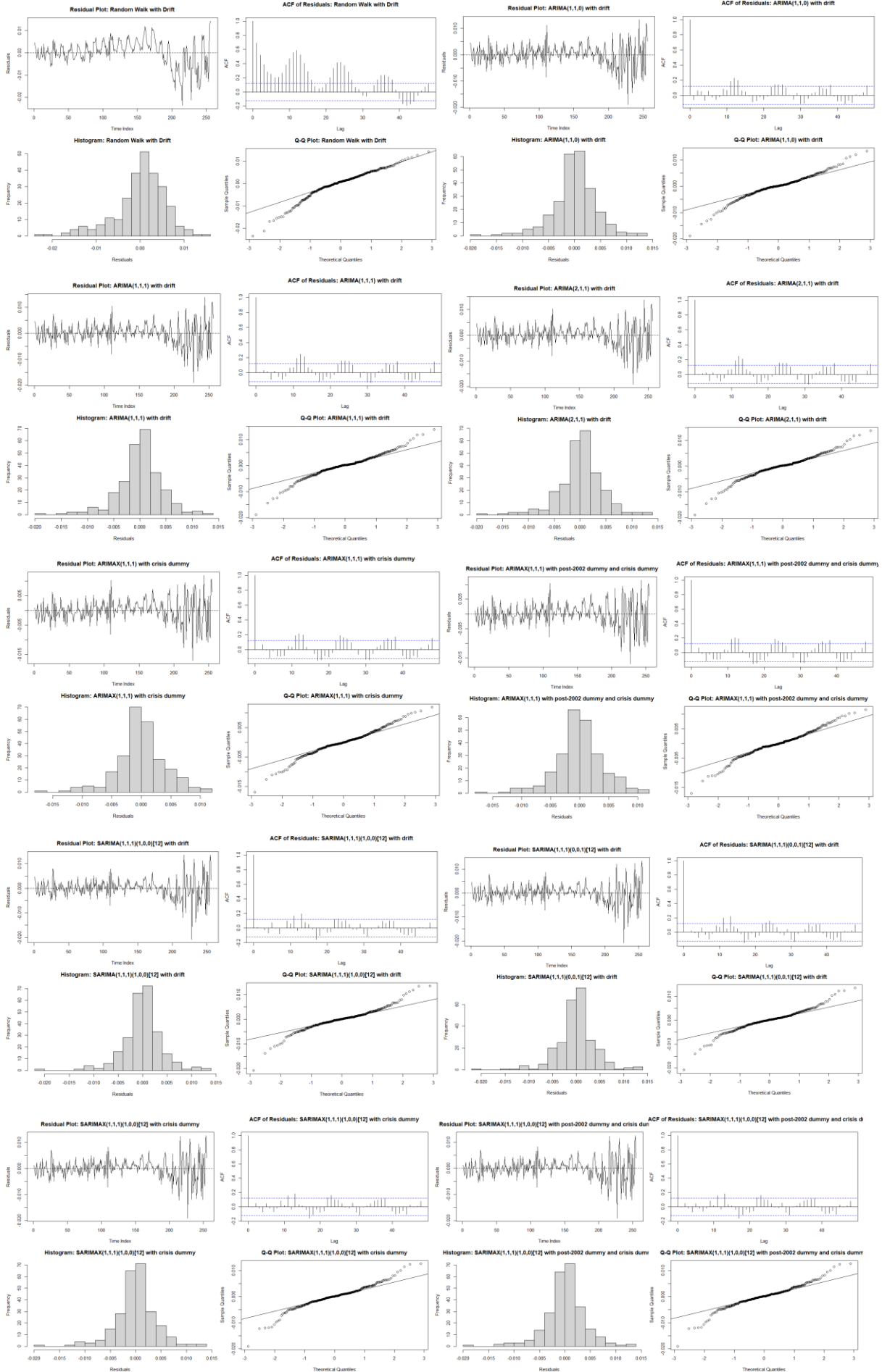


圖 7.1: 候選模型殘差圖、殘差 ACF 圖、直方圖、QQ-plot

Model	Max Abs Residual ACF	Ljung-Box Statistic	p-value	Decision
SARIMA(1,1,1)(1,0,0)[12] with drift	0.1956	51.229	0.000147	Retained
SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy	0.1837	51.458	0.0000797	Retained
SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy	0.1833	51.537	0.0000441	Retained
SARIMA(1,1,1)(0,0,1)[12] with drift	0.221	63.114	0.00000232	Retained
ARIMA(1,1,0) with drift	0.228	64.279	0.00000509	Not retained
ARIMA(2,1,1) with drift	0.247	73.113	< 0.000001	Not retained
ARIMA(1,1,1) with drift	0.2472	75.309	< 0.000001	Not retained
ARIMAX(1,1,1) with crisis dummy	0.2128	83.401	< 0.000001	Not retained
ARIMAX(1,1,1) with post- 2002 dummy and crisis dummy	0.2064	84.299	< 0.000001	Not retained
Random Walk with Drift	0.6913	885.039	< 0.000001	Not retained

表 7.1: 候選模型最大 ACF 絕對值、Ljung box test 重要數據

7.1 殘差時間數列圖 Residual Plot

所有候選模型皆先進行殘差視覺化檢查。由圖 7.1 殘差時間序列圖可觀察到，多數模型殘差大致圍繞 0 上下波動，未呈現明顯長期趨勢，表示模型已移除原始 HPI 中主要的長期趨勢成分。然而，在樣本後段仍可見較大殘差波動，尤其接近金融危機後的期間，殘差振幅明顯增加。表示 HPI 在特殊經濟時期仍具有較高不穩定性，單純 ARIMA 類模型較難以完全捕捉所有房市衝擊。

Random Walk with Drift 的殘差存在長期趨勢與明顯時間相依結構，表示其過於簡單，無法充分描述 HPI 動態變化。SARIMA 與 SARIMAX 模型的殘差波動相對較小，顯示加入 12 期相關結構後，模型對 HPI 月成長率的動態描述較佳。

7.2 殘差 ACF 圖 (ACF of Residuals)

由圖 7.1 殘差 ACF 結果可見，非季節 ARIMA 模型與 Random Walk with Drift 仍殘留較明顯的自我相關。其中 Random Walk with Drift 的 maximum absolute residual ACF 達 0.6913，顯示其殘差仍保留強烈時間結構。相較之下，SARIMA 與 SARIMAX 模型的 maximum absolute residual ACF 約落在 0.18 至 0.22 之間，明顯低於 Random Walk 非季節 ARIMA 模型，表示季節結構有助於降低殘差 ACF。

由表 7.1，根據前述保留機制，SARIMA 類模型的 Ljung-Box statistic 較小且 residual ACF 相對較低，將保留至預測環節進行比較，其餘模型雖可能在部分參數解釋上有意義，但殘差自相關較明顯，不作為後續主要預測模型。

7.3 常態性檢查 (Normality check)

使用 histogram 與 Q-Q plot 檢查殘差常態性。由圖 7.1，從所有模型的 histogram 可見，殘差大多集中在 0 附近，表示模型整體配適並未產生明顯系統性偏誤。然而，Q-Q plot 顯示多數模型在兩端尾部偏離常態直線，表示殘差並非完全符合常態分布。

此結果對 HPI 資料而言並不意外。住宅價格指數受到金融危機、房市循環與總體經濟衝擊影響，容易出現尾端較厚或極端波動。因此，後續預測區間仍可作為不確定性的參考，但在解釋時應注意尾端風險，尤其是在房市轉折或經濟衝擊期間，實際 HPI 可能偏離模型根據常態假設所給出的預測區間。

綜合殘差圖、殘差 ACF、Ljung-Box test 與常態性檢查結果，所有模型皆未完全達到 white-noise residual 的理想狀態，但季節模型整體表現相對較佳。因此，本研究保留下列模型進入下章進行樣本外預測比較：

- SARIMA(1,1,1)(1,0,0)[12] with drift
- SARIMA(1,1,1)(0,0,1)[12] with drift
- SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy
- SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy

8. 樣本外預測 (Forecasting)

使用第七章殘差診斷之 4 組保留模型進行樣本外預測。預測期間設定為 12 個月，測試資料期間為 2012 年 5 月至 2013 年 4 月。所有模型預測結果皆轉換回原始 HPI 尺度，與測試期間實際 HPI 比較，同時搭配 80% 與 95% prediction intervals 呈現預測不確定性，結果如圖 8.1:

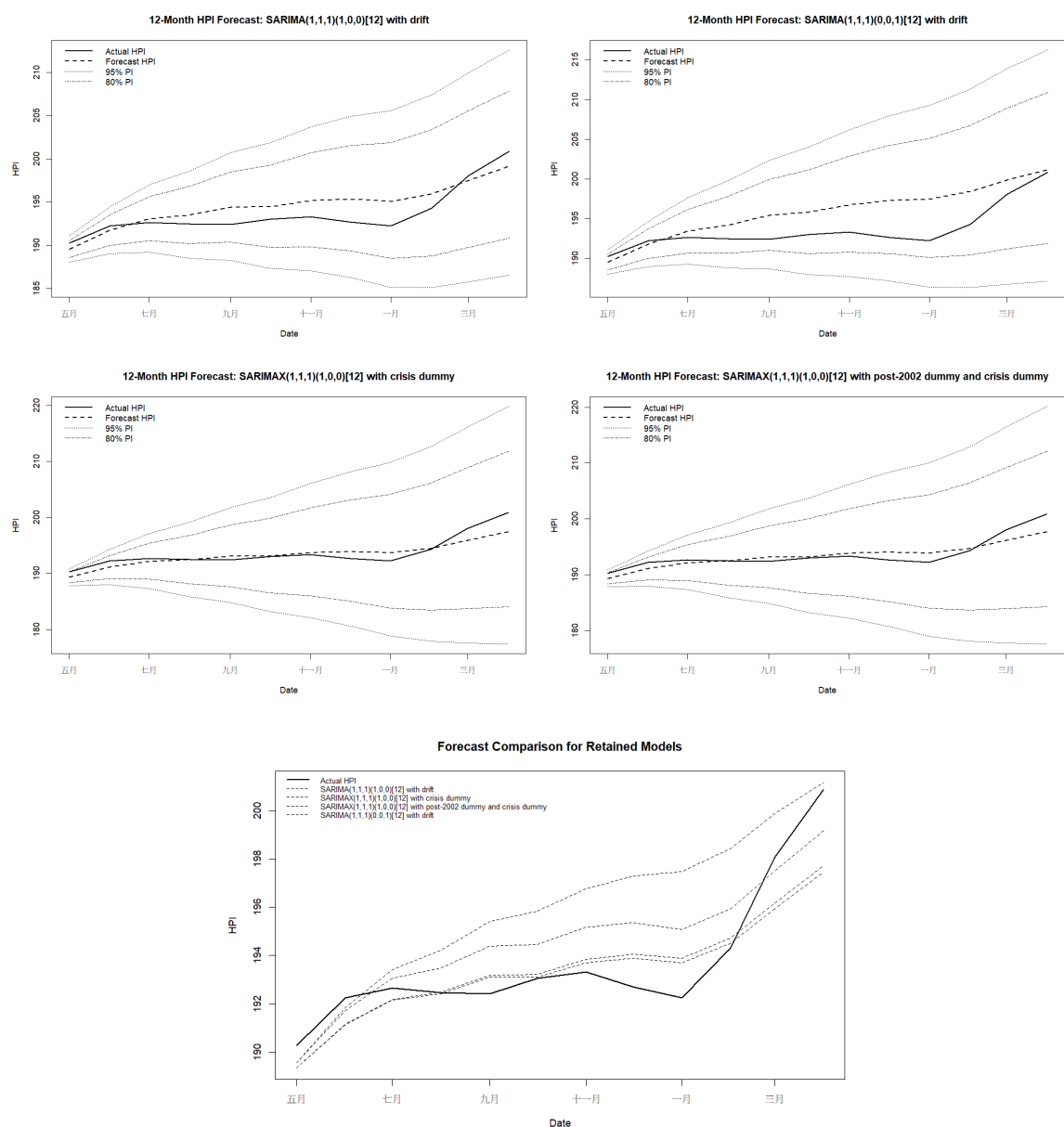


圖 8.1: 候選模型預測結果與比較

預測結果顯示，各模型皆預測 HPI 在測試期間呈緩步上升趨勢，與 2011 年後 HPI 逐漸回升的資料型態一致。從預測圖可見，實際 HPI 在多數期間皆落在 80% 或 95% prediction intervals 內，表示模型對短期 HPI 走勢的不確定性範圍設定大致合理。不過實際 HPI 在測試期後段上升速度明顯加快，多數模型雖能捕捉上升方向，但對後段回升幅度仍略有低估。就預測誤差而言，SARIMAX models 表現較佳，其中 SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy 的 RMSE 最低，為 1.3333；SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy 的 MAE 與 MAPE 則略低，分別為 1.0093 與 0.5160%。相較之下，SARIMA with drift 的預測誤差較大，表示僅使用固定 drift 捕捉平均成長率較難描述金融危機後房市復甦階段的變化。整體而言，加入 crisis dummy 與 12 期季節結構的 SARIMAX 較能反映金融危機 HPI 短期復甦走勢。Prediction intervals 隨著預測期數增加而逐漸變寬，表示時間越

往後，不確定性越高；因此，預測結果可作為短期房市趨勢參考，但在解釋後段快速回升或可能的結構轉折時仍需保持謹慎。

9. 模型比較 (Model Comparison)

綜合第六章模型估計、第七章殘差診斷與第八章樣本外預測結果，比較四個最終候選模型。相關數值列於表 9.1:

Model	AICc	BIC	Ljung-Box p-value	RMSE	MAE	MAPE
SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy	-2081.6	-2060.7	7.97E-05	1.3769	1.0093	0.52%
SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy	-2079.6	-2055.2	4.41E-05	1.3333	1.0342	0.53%
SARIMA(1,1,1)(1,0,0)[12] with drift	-2078.3	-2060.9	0.000147	1.6411	1.4419	0.74%
SARIMA(1,1,1)(0,0,1)[12] with drift	-2071	-2053.5	2.3E-06	2.9094	2.407	1.25%

表 9.1: 候選模型採用參考數值

由表 9.1 可見，SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy 具有最低 AICc，表示其在考慮模型複雜度後具有最佳的 in-sample fit。雖然 SARIMA(1,1,1)(1,0,0)[12] with drift 的 BIC 略低，代表其在較強懲罰模型複雜度的標準下稍具優勢，但該模型的樣本外 RMSE、MAE 與 MAPE 皆高於 SARIMAX 模型。因此，若同時考慮配適能力與預測表現，SARIMAX 模型較具優勢。

在預測表現方面，SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy 的 RMSE 最低，為 1.3333，表示其整體平方誤差最小。然而，SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy 的 MAE 與 MAPE 最低，分別為 1.0093 與 0.5160。兩個 SARIMAX 模型的預測表現非常接近，且皆明顯優於兩個 SARIMA with drift 模型。

從模型解釋性來看，SARIMAX with crisis dummy 較為簡潔且具有明確經濟意義。第七章估計結果顯示，crisis dummy 為負且顯著，表示 2007–2010 年金融危機期間 HPI 月成長率明顯下降。相較之下，post-2002 dummy 在同時加入 crisis dummy 後並未呈現明顯顯著性，因此加入 post-2002 dummy 雖使 RMSE 略微降低，但增加了模型複雜度，且其額外經濟解釋力有限。

殘差診斷方面，四個模型殘差皆未完全通過 Ljung-Box test，表示仍有部分殘差自我相關。然而，SARIMA 與 SARIMAX 類模型的 Ljung-Box statistic 與 maximum absolute residual ACF 明顯優於 random Walk 與非季節 ARIMA 模型，因此已是候選模型中殘差行為相對較佳者，也表示加入 12 期季節結構對捕捉 HPI 的時間相依結構有幫助。

綜合各項比較，雖然加入 post-2002 dummy 的 SARIMAX 模型具有最低 RMSE，但其改善幅度有限，且 post-2002 dummy 的解釋力較弱，不選為最終模型。因此，本文最終選擇

SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy 作為最終模型。此模型不僅具有最低 AICc，也在 MAE 與 MAPE 上表現最佳，且 crisis dummy 提供清楚經濟解釋，能夠捕捉金融危機期間 HPI 成長率下滑的結構性影響。

10. 結論 (Conclusion)

本文以美國住宅價格指數 HPI 作為分析對象，建立時間數列模型描述其歷史變化並預測短期走勢。由 EDA 可知，HPI 在樣本期間具明顯長期趨勢，1991 年後逐漸上升，2002 年後成長速度加快，並於 2007 至 2010 年金融危機期間出現明顯下跌與較大波動。經 log 轉換與一階差分後，序列趨勢性明顯降低，但仍可觀察到短期持續性、12 期相關結構，以及金融危機期間的負向衝擊。

經模型估計、殘差診斷與樣本外預測比較，本文最終選擇 SARIMAX(1,1,1)(1,0,0)[12]with crisis dummy 作為最終模型。此模型同時納入 ARMA 動態結構、12 期 seasonal AR 項與金融危機 dummy，能捕捉 HPI 月成長率的短期持續性、年度相關結構，以及 2007–2010 年金融危機期間造成的結構性下滑。相較於 SARIMA with drift 模型，SARIMAX 模型在樣本外預測表現上較佳，且 crisis dummy 具有明確經濟解釋。

在預測結果方面，模型預測 2012 年 5 月至 2013 年 4 月的 HPI 大致呈現緩步上升趨勢，與金融危機後住宅價格逐漸回升的資料型態一致。實際 HPI 皆落在 80% 或 95% prediction intervals 內，表示模型對短期預測不確定性的衡量大致合理。不過，測試期後段 HPI 上升速度較快，模型仍略有低估，顯示 ARIMA 類模型對快速復甦或結構轉折的捕捉能力仍有限。在可改進的方向上，Larson (2011) 指出，多變量且具有經濟理論基礎的模型，在預測房市轉折時可能較具優勢。然而本文主要使用單變量 HPI 建模，未納入利率、所得、租金、信用條件或其他總體經濟變數，因此模型對房市轉折點與外部衝擊的解釋仍有限。後續研究可進一步考慮多變量模型或 structural time series model，以更完整捕捉住宅價格變動的經濟因素與結構變化。

參考文獻 References

- Larson, W. D. (2011). *Evaluating alternative methods of forecasting house prices: A post-crisis reassessment*. Research Program on Forecasting Working Paper No. 2010-004, The George Washington University.
- Shumway and Stoffer. (2025). *Time Series Analysis and its Applications with R Examples*.
- Tai, Y.-C. (2026). *Time Series Analysis: 304028011/354707001, Lecture Notes 1-11*.
[Course lecture notes] Department of Statistics, National Chengchi University.
- Texas Tech University. (n.d.). *Time Series Data Sets: Economics/Finance*.

附錄 A: 程式碼 (Python+R)

附錄提供本文分析所使用之程式碼，包括資料清理、EDA、資料轉換、建模流程。主文中所有表格與圖形皆可由以下程式碼重現。

A.1 Data Cleaning (Python)

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

In [2]: df = pd.read_csv("data/raw/monthly-housing.csv")
df.head(3)

Out[2]:
```

	date	hpi	numsold
0	1/1/1991	100.00	30
1	2/1/1991	100.48	40
2	3/1/1991	100.74	51

```
Remove null value

In [3]: print(df.shape)
print(df[["date", "hpi"]].isnull().sum())

(280, 3)
date      12
hpi       12
dtype: int64

In [4]: df_clean = df.dropna(subset=["date", "hpi"]).copy()
print(df_clean[["date", "hpi"]].isnull().sum())
print(df_clean.shape)

date      0
hpi       0
dtype: int64
(268, 3)

Change date format

In [5]: df_clean["date"] = pd.to_datetime(df_clean["date"])
df_clean = df_clean.sort_values("date").reset_index(drop=True)
df_clean = df_clean.set_index("date")
df_clean = df_clean.asfreq("MS") # set as the beginning of the month

In [6]: print(df_clean.index)
print(df_clean.head(3))
print(df_clean.tail(3))

DatetimeIndex(['1991-01-01', '1991-02-01', '1991-03-01', '1991-04-01',
                '1991-05-01', '1991-06-01', '1991-07-01', '1991-08-01',
                '1991-09-01', '1991-10-01',
                ...,
                '2012-07-01', '2012-08-01', '2012-09-01', '2012-10-01',
                '2012-11-01', '2012-12-01', '2013-01-01', '2013-02-01',
                '2013-03-01', '2013-04-01'],
              dtype='datetime64[ns]', name='date', length=268, freq='MS')

           hpi  numsold
date
1991-01-01  100.00     30
1991-02-01  100.48     40
1991-03-01  100.74     51
...
date
2013-02-01  194.31     36
2013-03-01  198.06     41
2013-04-01  200.87     43

Visualize df_clean

In [7]: plt.figure(figsize=(10, 4))

plt.plot(
    df_clean.index,
    df_clean["hpi"],
    linewidth=0.8
)

y_min = df_clean["hpi"].min()
y_max = df_clean["hpi"].max()
padding = (y_max - y_min) * 0.03

plt.ylim(y_min - padding, y_max + padding)

plt.title("Housing Price Index Over Time")
plt.xlabel("Date")
plt.ylabel("HPI")
plt.grid(True, alpha=0.3)

plt.show()
```



save df_clean

```
In [8]: df_clean.to_csv("data/processed/monthly_housing_clean.csv", index=True)
```

train test split

```
In [9]: train = df_clean.iloc[:-12].copy()
test = df_clean.iloc[-12:].copy()

print("Train shape:", train.shape)
print("Test shape:", test.shape)

print(train.head(3))
print(train.tail(3))

print(test.head(3))
print(test.tail(3))
```

Train shape: (256, 2)

Test shape: (12, 2)

date	hpi	numsold
1991-01-01	100.00	30
1991-02-01	100.48	40
1991-03-01	100.74	51
...		

date	hpi	numsold
2012-02-01	181.36	30
2012-03-01	184.15	34
2012-04-01	187.26	34

date	hpi	numsold
2012-05-01	190.29	35
2012-06-01	192.25	34
2012-07-01	192.66	33
...		

date	hpi	numsold
2013-02-01	194.31	36
2013-03-01	198.06	41
2013-04-01	200.87	43

```
In [10]: train.to_csv("data/processed/monthly_housing_train.csv", index=True)
test.to_csv("data/processed/monthly_housing_test.csv", index=True)
```

A2. EDA (Python)

```
In [11]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
```

```
In [14]: df_train = pd.read_csv("data/processed/monthly_housing_train.csv")
df_train['date'] = pd.to_datetime(df_train['date'])
df_train = df_train.set_index('date')
df_train.head(3)
```

```
Out[14]:
```

	hpi	numsold
date		
1991-01-01	100.00	30
1991-02-01	100.48	40
1991-03-01	100.74	51

Visualize train data

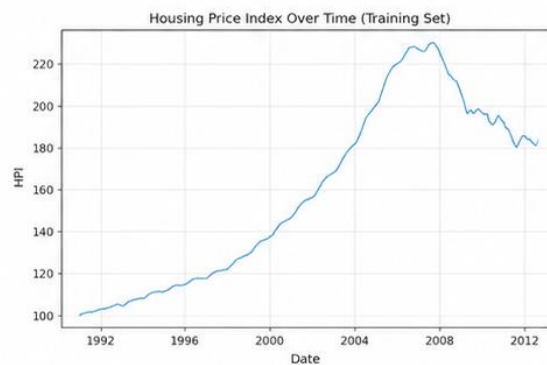
```
In [15]: plt.figure(figsize=(10, 4))

plt.plot(
    df_train.index,
    df_train['hpi'],
    linewidth=0.8
)

plt.title("Housing Price Index Over Time (Training Set)")
plt.xlabel("Date")
plt.ylabel("HPI")
plt.grid(True, alpha=0.3)

ax = plt.gca()
ax.xaxis.set_major_locator(mdates.YearLocator(4)) # 每 4 年一格
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

plt.tight_layout()
plt.show()
```



```
In [16]: summary_stats = pd.DataFrame({
    "HPI": [
        df_train['hpi'].mean(),
        df_train['hpi'].std(),
        df_train['hpi'].min(),
        df_train['hpi'].quantile(0.25),
        df_train['hpi'].median(),
        df_train['hpi'].quantile(0.75),
        df_train['hpi'].max(),
        df_train['hpi'].std() / df_train['hpi'].mean()
    ],
    index=[
        "Mean",
        "Standard deviation",
        "Minimum",
        "First quartile",
        "Median",
        "Third quartile",
        "Maximum",
        "CV"
    ]
})

summary_stats_rounded = summary_stats.round(3)
summary_stats_rounded
```

```
Out[16]:
```

	HPI
Mean	157.199
Standard deviation	42.155
Minimum	100.000
First quartile	116.117
Median	154.290
Third quartile	195.122
Maximum	228.710
CV	0.268

```
In [14]: df_train = pd.read_csv("data/processed/monthly_housing_train.csv")
df_train['date'] = pd.to_datetime(df_train['date'])
df_train = df_train.set_index('date')
df_train.head(3)
```

```
Out[14]:
```

	hpi	numsold
date		
1991-01-01	100.00	30
1991-02-01	100.48	40
1991-03-01	100.74	51

```
In [15]: plt.figure(figsize=(10, 4))

plt.plot(
    df_train.index,
    df_train['numsold'],
    linewidth=0.8
)

plt.title("Number of Houses Sold Over Time (Training Set)")
plt.xlabel("Date")
plt.ylabel("Number of Houses Sold")
plt.grid(True, alpha=0.3)

ax = plt.gca()
ax.xaxis.set_major_locator(mdates.YearLocator(4)) # 每 4 年一格
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

plt.tight_layout()
plt.show()
```



A.3 Stationary Inspection (Python)

```
In [2]: !pip install statsmodels
Requirement already satisfied: statsmodels in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (0.14.0)
Requirement already satisfied: numpy<1.22.3 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (1.24.3)
Requirement already satisfied: scipy<1.9.2,>1.8 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (1.10.0)
Requirement already satisfied: pandas<2.1.0,>1.4 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (1.5.3)
Requirement already satisfied: patsy<0.5.6 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (0.5.6)
Requirement already satisfied: packaging<21.3 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (23.1)
Requirement already satisfied: python-dateutil<2.8.2 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (2.8.2)
Requirement already satisfied: pytz<2020.1 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (2020.1)
Requirement already satisfied: tzdata<2022.1 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (2022.1)
Requirement already satisfied: six<1.5 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (1.16.0)

[notice] A new release of pip available: 22.3.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [3]: import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
```

```
In [2]: df = pd.read_csv("data\processed\monthly_housing_train.csv")
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date")
df = df.set_index("date")
y = df["hpi"].dropna()

# rolling mean & rolling standard deviation
y_rolling_mean_12 = y.rolling(window=12).mean()
y_rolling_std_12 = y.rolling(window=12).std()
```

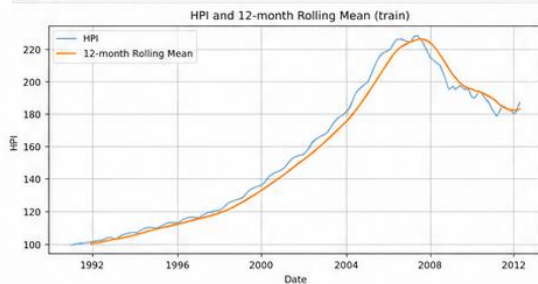
Stationary inspection

12 month rolling mean

```
In [3]: plt.figure(figsize=(10, 5))

plt.plot(y.index, y, label="HPI", alpha=0.6)
plt.plot(y_rolling_mean_12.index, y_rolling_mean_12,
         label="12-month Rolling Mean", linewidth=2)

plt.title("HPI and 12-month Rolling Mean (train)")
plt.xlabel("Date")
plt.ylabel("HPI")
plt.legend()
plt.grid(True)
plt.show()
```

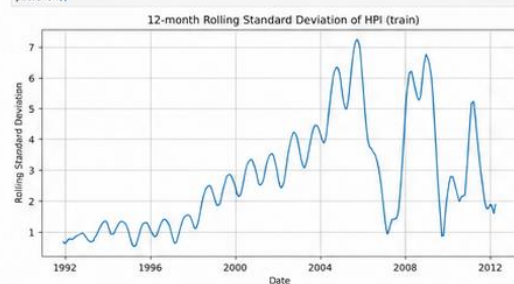


12 month rolling std

```
In [5]: plt.figure(figsize=(10, 5))

plt.plot(y_rolling_std_12.index, y_rolling_std_12)

plt.title("12-month Rolling Standard Deviation of HPI (train)")
plt.xlabel("Date")
plt.ylabel("Rolling Standard Deviation")
plt.grid(True)
plt.show()
```



ACF plot

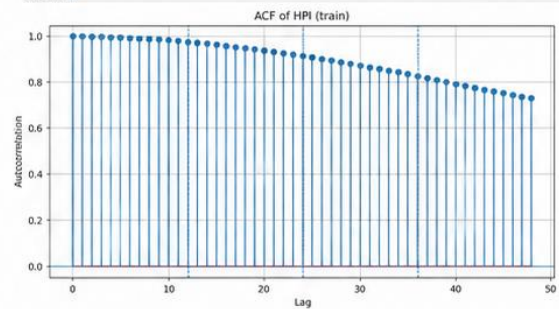
```
In [6]: max_lag = 48
acf_values = [y.autocorr(lag=lag) for lag in range(max_lag + 1)]

plt.figure(figsize=(10, 5))
plt.stem(range(max_lag + 1), acf_values)

for lag in [12, 24, 36]:
    plt.axvline(x=lag, linestyle="--", linewidth=1)

plt.axhline(y=0, linewidth=1)

plt.title("ACF of HPI (train)")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")
plt.grid(True)
plt.show()
```



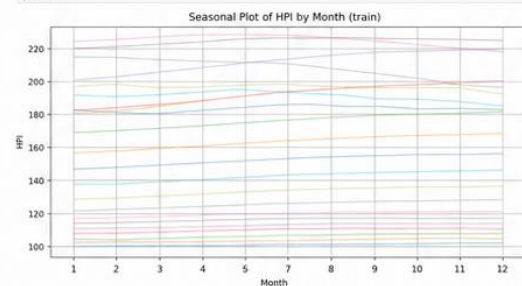
monthly seasonal plot

```
In [7]: season_df = y.to_frame(name="hpi")
season_df["year"] = season_df.index.year
season_df["month"] = season_df.index.month

plt.figure(figsize=(10, 5))

for year, group in season_df.groupby("year"):
    plt.plot(group["month"], group["hpi"], alpha=0.35)

plt.title("Seasonal Plot of HPI by Month (train)")
plt.xlabel("Month")
plt.ylabel("HPI")
plt.xticks(range(1, 13))
plt.grid(True)
plt.show()
```



ADF test

```
In [8]: # ADF test
for reg in ["c", "ct"]:
    result = adfuller(y, regression=reg, autolag="AIC")
    print(f"ADF test regression = {reg}")
    print("-----")
    print(f"ADF Statistic: (result[0]:.4f)")
    print(f"P-value: (result[1]:.4f)")
    print(f"Used Lag: (result[2]:.4f)")
    print(f"Critical Values:")
    for key, value in result[4].items():
        print(f"({key}): (value:.4f)")
```

```
ADF test regression = c
-----
ADF Statistic: -1.1101
p-value: 0.7110
Used Lag: 16
Critical Values:
1%: -3.4580
5%: -2.8737
10%: -2.5733

ADF test regression = ct
-----
ADF Statistic: -2.8951
p-value: 0.1638
Used Lag: 16
Critical Values:
1%: -3.9972
5%: -3.4290
10%: -3.1379
```


A.4 Transformation and Differencing (Python)

```
In [18]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import zivot_andrews
```

```
In [7]: df = pd.read_csv("data/processed/monthly_housing_clean.csv", parse_dates=["date"])
df = df.sort_values("date").set_index("date")
df.head(3)
```

```
Out[7]:
```

	hpi	numsold
date		
1991-01-01	100.00	30
1991-02-01	100.48	40
1991-03-01	100.74	51

```
In [ ]: # transformation
df["log_hpi"] = np.log(df["hpi"])
df["diff_log_hpi"] = df["log_hpi"].diff()

# split
train = df.loc[: "2012-04"].copy()
test = df.loc["2012-05" :].copy()

# Log-diff series
diff_log_train = train[["hpi", "log_hpi", "diff_log_hpi"]].dropna()
diff_log_test = test[["hpi", "log_hpi", "diff_log_hpi"]].copy()

# save
train.to_csv("data/processed/monthly_housing_train.csv")
test.to_csv("data/processed/monthly_housing_test.csv")
```

TS plot

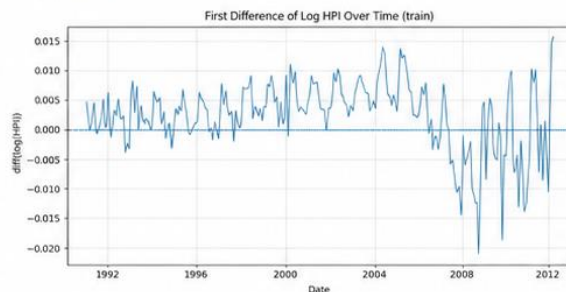
```
In [11]: plt.figure(figsize=(12, 5))

plt.plot(
    diff_log_train.index,
    diff_log_train["diff_log_hpi"],
    linewidth=1
)

plt.axhline(0, linestyle="--", linewidth=1)

plt.title("First Difference of Log HPI Over Time (train)")
plt.xlabel("Date")
plt.ylabel("diff(log(HPI))")
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
In [13]: import matplotlib.dates as mdates
```

```
diff_log_train["rolling_mean_12"] = (
    diff_log_train["diff_log_hpi"]
    .rolling(window=12)
    .mean()
)

diff_log_train["rolling_std_12"] = (
    diff_log_train["diff_log_hpi"]
    .rolling(window=12)
    .std()
)
```

```
In [12]: plt.figure(figsize=(12, 5))

plt.plot(
    diff_log_train.index,
    diff_log_train["diff_log_hpi"],
    label="diff(log(HPI))",
    alpha=0.5
)

plt.plot(
    diff_log_train.index,
    diff_log_train["rolling_mean_12"],
    label="12-month Rolling Mean",
    linewidth=2
)

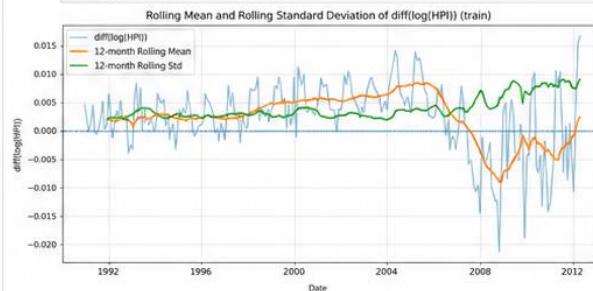
plt.plot(
    diff_log_train.index,
    diff_log_train["rolling_std_12"],
    label="12-month Rolling Std",
    linewidth=2
)

plt.axhline(0, linestyle="--", linewidth=1)

plt.title("Rolling Mean and Rolling Standard Deviation of diff(log(HPI)) (train)")
plt.xlabel("Date")
plt.ylabel("diff(log(HPI))")
plt.legend()
plt.grid(True, alpha=0.3)

ax = plt.gca()
ax.xaxis.set_major_locator(mdates.YearLocator(4))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

plt.tight_layout()
plt.show()
```



```
In [20]: fig, ax = plt.subplots(figsize=(10, 5))

plot_acf(
    diff_log_train["diff_log_hpi"],
    lags=50,
    ax=ax
)

ax.set_title("ACF of First Difference of Log HPI (train)")
ax.set_xlabel("Lag")
ax.set_ylabel("Autocorrelation")
ax.grid(True, alpha=0.3)

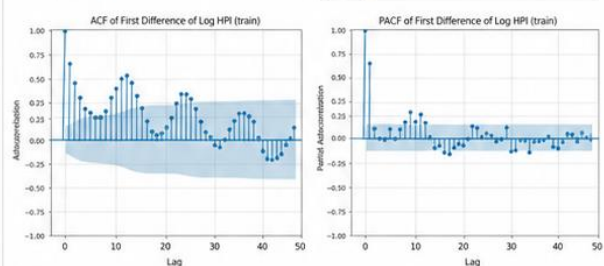
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(10, 5))

plot_pacf(
    diff_log_train["diff_log_hpi"],
    lags=50,
    ax=ax,
    method="yw"
)

ax.set_title("PACF of First Difference of Log HPI (train)")
ax.set_xlabel("Lag")
ax.set_ylabel("Partial Autocorrelation")
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
In [15]: adf_result = adfuller(diff_log_train["diff_log_hpi"])
```

```
print("ADF Test for diff(log(HPI))")
print("-----")
print(f"ADF Statistic: {adf_result[0]:.4f}")
print(f"p-value: {adf_result[1]:.4f}")
print(f"Number of Lags Used: {adf_result[2]}")
print(f"Number of Observations: {adf_result[3]}")

print("Critical Values:")
for key, value in adf_result[4].items():
    print(f"({key}): {value:.4f}")

ADF Test for diff(log(HPI))
-----
ADF Statistic: -2.4212
p-value: 0.1358
Number of Lags Used: 16
Number of Observations: 238
Critical Values:
1%: -3.4581
5%: -2.8738
10%: -2.5733
```

A.5 Candidate Model (R)

```
# =====
# Function
# =====

check_required_packages <- function(packages) {
  for (pkg in packages) {
    if (!requireNamespace(pkg, quietly = TRUE)) {
      stop(paste0("Package '", pkg, "' is required. Please install it first."))
    }
  }
}

load_housing_data <- function(file_path) {
  df <- read.csv(file_path, stringsAsFactors = FALSE)

  required_cols <- c("date", "hpi")
  missing_cols <- setdiff(required_cols, names(df))

  if (length(missing_cols) > 0) {
    stop(paste("Missing required columns:", paste(missing_cols, collapse = ",
")))
  }

  df$date <- as.Date(df$date)
  df <- df[order(df$date), ]

  if (!"log_hpi" %in% names(df)) {
    df$log_hpi <- log(df$hpi)
  }

  if (!"diff_log_hpi" %in% names(df)) {
    df$diff_log_hpi <- c(NA, diff(df$log_hpi))
  }

  return(df)
}

make_monthly_ts <- function(values, dates) {
  start_year <- as.integer(format(min(dates), "%Y"))
  start_month <- as.integer(format(min(dates), "%m"))

  ts(
    values,
    start = c(start_year, start_month),
    frequency = 12
  )
}

add_dummy_variables <- function(df) {
  df$post2002 <- ifelse(df$date >= as.Date("2002-01-01"), 1, 0)

  df$crisis <- ifelse(
    df$date >= as.Date("2007-01-01") &
    df$date <= as.Date("2010-12-01"),
    1,
    0
  )

  return(df)
}

prepare_model_inputs <- function(df) {
  df <- add_dummy_variables(df)

  log_hpi_ts <- make_monthly_ts(
    values = df$log_hpi,
    dates = df$date
  )

  df_diff <- df[!is.na(df$diff_log_hpi), ]

  diff_log_hpi_ts <- make_monthly_ts(
    values = df_diff$diff_log_hpi,
    dates = df_diff$date
  )

  xreg_crisis <- as.matrix(df_diff[, c("crisis"), drop = FALSE])
  xreg_both <- as.matrix(df_diff[, c("post2002", "crisis"), drop = FALSE])

  list(
    df = df,
    df_diff = df_diff,
    log_hpi_ts = log_hpi_ts,
    diff_log_hpi_ts = diff_log_hpi_ts,
    xreg_crisis = xreg_crisis,
    xreg_both = xreg_both
  )
}

fit_candidate_models <- function(inputs) {
  models <- list(
    # Baseline model
    "Random Walk with Drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(0, 1, 0),
      include.drift = TRUE,
      method = "ML"
    ),
    # ARIMA models with drift
    "ARIMA(1,1,0) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 0),
      include.drift = TRUE,
      method = "ML"
    ),
    "ARIMA(1,1,1) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      include.drift = TRUE,
      method = "ML"
    ),
    "ARIMA(2,1,1) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(2, 1, 1),
      include.drift = TRUE,
      method = "ML"
    ),
    # SARIMA models with drift
    "SARIMA(1,1,1)(1,0,0)[12] with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      seasonal = list(order = c(1, 0, 0), period = 12),
      include.drift = TRUE,
      method = "ML"
    ),
    "SARIMA(1,1,1)(0,0,1)[12] with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      seasonal = list(order = c(0, 0, 1), period = 12),
      include.drift = TRUE,
      method = "ML"
    ),
    # ARIMAX models with dummy variables
    # These models use dummy variables to capture changes in average growth.
    "ARIMAX(1,1,1) with crisis dummy" = forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      xreg = inputs$xreg_crisis,
      include.mean = TRUE,
      method = "ML"
    ),
    "ARIMAX(1,1,1) with post-2002 dummy and crisis dummy" = forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      xreg = inputs$xreg_both,
      include.mean = TRUE,
      method = "ML"
    ),
    # SARIMAX models with dummy variables
    # These models add the 12-month dependence structure and use dummy
    # variables
    # to capture changes in average growth.
    "SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy" = forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      seasonal = list(order = c(1, 0, 0), period = 12),
      xreg = inputs$xreg_crisis,
      include.mean = TRUE,
      method = "ML"
    ),
    "SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy" =
    forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      seasonal = list(order = c(1, 0, 0), period = 12),
      xreg = inputs$xreg_both,
      include.mean = TRUE,
      method = "ML"
    )
  )

  return(models)
}

extract_ic_table <- function(models) {
  table <- do.call(
    rbind,
    lapply(names(models), function(model_name) {
      fit <- models[[model_name]]

      data.frame(
        Model = model_name,
        AIC = as.numeric(AIC(fit)),
        AICc = as.numeric(AICc(fit)),
        BIC = as.numeric(BIC(fit)),
        row.names = NULL
      )
    })
  )

  table <- table[order(table$AICc), ]

  return(table)
}

extract_parameter_table <- function(models) {
  table <- do.call(
    rbind,
    lapply(names(models), function(model_name) {
      fit <- models[[model_name]]
      coefs <- stats::coef(fit)

      if (length(coefs) == 0) {
        return(NULL)
      }

      se <- rep(NA_real_, length(coefs))
      names(se) <- names(coefs)

      if (!is.null(fit$var.coef)) {

```

```

        available_se <- sqrt(diag(fit$var.coef))
        se[names(available_se)] <- available_se
    }

    z_value <- coefs / se
    p_value <- 2 * (1 - stats::pnorm(abs(z_value)))

    data.frame(
        Model = model_name,
        Parameter = names(coefs),
        Estimate = as.numeric(coefs),
        Std_Error = as.numeric(se),
        z_value = as.numeric(z_value),
        p_value = as.numeric(p_value),
        row.names = NULL
    )
})
}

return(table)
}

print_model_tables <- function(ic_table, parameter_table) {
    cat("\n===== Information Criteria Table =====\n")
    print(ic_table, row.names = FALSE, digits = 5)

    cat("\n===== Parameter Estimation Table =====\n")
    print(parameter_table, row.names = FALSE, digits = 5)
}

print_key_summaries <- function(models) {
    cat("\n===== ARIMA(1,1,1) with drift Summary =====\n")
    print(summary(models[["ARIMA(1,1,1) with drift"]]))

    cat("\n===== ARIMAX(1,1,1) with post-2002 dummy and crisis dummy Summary  
===== \n")
    print(summary(models[["ARIMAX(1,1,1) with post-2002 dummy and crisis dummy"]]))

    cat("\n===== SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy  
Summary ===== \n")
    print(summary(models[["SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy"]]))
}

run_model_estimation <- function(file_path) {
    set.seed(123)

    check_required_packages(c("forecast"))

    df <- load_housing_data(file_path)
    inputs <- prepare_model_inputs(df)
    models <- fit_candidate_models(inputs)

    ic_table <- extract_ic_table(models)
    parameter_table <- extract_parameter_table(models)

    print_model_tables(
        ic_table = ic_table,
        parameter_table = parameter_table
    )

    print_key_summaries(models)

    results <- list(
        data = df,
        inputs = inputs,
        models = models,
        ic_table = ic_table,
        parameter_table = parameter_table
    )

    return(invisible(results))
}

# =====
# Main
# =====

# Run the analysis and print results in the console.
invisible(run_model_estimation(
    file_path = "C:/Users/use/Desktop/NCCU_STAT/Time
Series/final/data/processed/monthly_housing_train.csv"
))

```

A.6 Model Diagnostics (R)

```
# ---- Function ----
check_required_packages <- function(packages) {
  for (pkg in packages) {
    if (!requireNamespace(pkg, quietly = TRUE)) {
      stop(paste0("Package '", pkg, "' is required. Please install it first."))
    }
  }
}

load_housing_data <- function(file_path) {
  df <- read.csv(file_path, stringsAsFactors = FALSE)

  required_cols <- c("date", "hpi")
  missing_cols <- setdiff(required_cols, names(df))

  if (length(missing_cols) > 0) {
    stop(paste("Missing required columns:", paste(missing_cols, collapse = ",
")))
  }

  df$date <- as.Date(df$date)
  df <- df[order(df$date), ]

  if (!"log_hpi" %in% names(df)) {
    df$log_hpi <- log(df$hpi)
  }

  if (!"diff_log_hpi" %in% names(df)) {
    df$diff_log_hpi <- c(NA, diff(df$log_hpi))
  }

  return(df)
}

make_monthly_ts <- function(values, dates) {
  start_year <- as.integer(format(min(dates), "%Y"))
  start_month <- as.integer(format(min(dates), "%n"))

  ts(
    values,
    start = c(start_year, start_month),
    frequency = 12
  )
}

add_dummy_variables <- function(df) {
  df$post2002 <- ifelse(df$date >= as.Date("2002-01-01"), 1, 0)

  df$crisis <- ifelse(
    df$date >= as.Date("2007-01-01") &
    df$date <= as.Date("2010-12-01"),
    1,
    0
  )

  return(df)
}

prepare_model_inputs <- function(df) {
  df <- add_dummy_variables(df)

  log_hpi_ts <- make_monthly_ts(
    values = df$log_hpi,
    dates = df$date
  )

  df_diff <- df[!is.na(df$diff_log_hpi), ]

  diff_log_hpi_ts <- make_monthly_ts(
    values = df_diff$diff_log_hpi,
    dates = df_diff$date
  )

  xreg_crisis <- as.matrix(df_diff[, c("crisis"), drop = FALSE])
  xreg_both <- as.matrix(df_diff[, c("post2002", "crisis"), drop = FALSE])

  list(
    df = df,
    df_diff = df_diff,
    log_hpi_ts = log_hpi_ts,
    diff_log_hpi_ts = diff_log_hpi_ts,
    xreg_crisis = xreg_crisis,
    xreg_both = xreg_both
  )
}

fit_candidate_models <- function(inputs) {
  models <- list(
    "Random Walk with Drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(0, 1, 0),
      include.drift = TRUE,
      method = "ML"
    ),
    "ARIMA(1,1,0) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 0),
      include.drift = TRUE,
      method = "ML"
    ),
    "ARIMA(1,1,1) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      include.drift = TRUE,
      method = "ML"
    ),
    "ARIMA(2,1,1) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(2, 1, 1),
      include.drift = TRUE,
      method = "ML"
    ),
    "SARIMA(1,1,1)(1,0,0)[12] with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      seasonal = list(order = c(1, 0, 0), period = 12),
      include.drift = TRUE,
      method = "ML"
    ),
    "SARIMA(1,1,1)(0,0,1)[12] with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      seasonal = list(order = c(0, 0, 1), period = 12),
      include.drift = TRUE,
      method = "ML"
    ),
    "ARIMAX(1,1,1) with crisis dummy" = forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      xreg = inputs$xreg_crisis,
      include.mean = TRUE,
      method = "ML"
    ),
    "ARIMAX(1,1,1) with post-2002 dummy and crisis dummy" = forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      xreg = inputs$xreg_both,
      include.mean = TRUE,
      method = "ML"
    ),
    "SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy" = forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      seasonal = list(order = c(1, 0, 0), period = 12),
      xreg = inputs$xreg_crisis,
      include.mean = TRUE,
      method = "ML"
    ),
    "SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy" =
    forecast::Arima(
      inputs$diff_log_hpi_ts,
      order = c(1, 0, 1),
      seasonal = list(order = c(1, 0, 0), period = 12),
      xreg = inputs$xreg_both,
      include.mean = TRUE,
      method = "ML"
    )
  )

  return(models)
}

run_model_estimation <- function(file_path) {
  set.seed(123)

  check_required_packages(c("forecast"))

  df <- load_housing_data(file_path)
  inputs <- prepare_model_inputs(df)
  models <- fit_candidate_models(inputs)

  list(
    data = df,
    inputs = inputs,
    models = models
  )
}

get_clean_residuals <- function(fit) {
  residuals <- stats::residuals(fit)
  residuals <- residuals[!is.na(residuals)]
  return(residuals)
}

get_max_abs_residual_acf <- function(fit, lag.max = 24) {
  residuals <- get_clean_residuals(fit)

  acf_values <- stats::acf(
    residuals,
    lag.max = lag.max,
    plot = FALSE
  )$acf

  acf_values <- acf_values[-1]

  max(abs(acf_values))
}

run_ljung_box_test <- function(fit, lag = 24) {
  residuals <- get_clean_residuals(fit)
  n_parameters <- length(stats::coef(fit))

  test_result <- stats::Box.test(
    residuals,
    lag = lag,
    type = "Ljung-Box",
    fitdf = n_parameters
  )

  list(
```



```

    statistic = as.numeric(test_result$statistic),
    p_value = as.numeric(test_result$p.value)
  }
}

diagnose_single_model <- function(model_name, fit, lag = 24, acf_lag_max = 24) {
  residuals <- get_clean_residuals(fit)
  ljung_box <- run_ljung_box_test(fit, lag = lag)
  max_abs_acf <- get_max_abs_residual_acf(fit, lag.max = acf_lag_max)

  data.frame(
    Model = model_name,
    Residual_Mean = mean(residuals),
    Residual_SD = stats::sd(residuals),
    Max_Abs_ACF_Lag_1_to_24 = max_abs_acf,
    Ljung_Box_Lag = lag,
    Ljung_Box_Statistic = ljung_box$statistic,
    Ljung_Box_p_value = ljung_box$p_value,
    row.names = NULL
  )
}

diagnose_all_models <- function(models, lag = 24, acf_lag_max = 24) {
  diagnostics_table <- do.call(
    rbind,
    lapply(names(models), function(model_name) {
      diagnose_single_model(
        model_name = model_name,
        fit = models[[model_name]],
        lag = lag,
        acf_lag_max = acf_lag_max
      )
    })
  )

  diagnostics_table <- diagnostics_table[order(
    diagnostics_table$Ljung_Box_Statistic,
    diagnostics_table$Max_Abs_ACF_Lag_1_to_24
  ), ]

  return(diagnostics_table)
}

select_models_by_relative_diagnostics <- function(diagnostics_table, top_n = 4) {
  head(diagnostics_table$Model, top_n)
}

add_retention_decision <- function(diagnostics_table, selected_model_names) {
  diagnostics_table$Diagnostics_Decision <- ifelse(
    diagnostics_table$Model %in% selected_model_names,
    "Retained for forecasting",
    "Not retained"
  )
}

return(diagnostics_table)
}

# Visualization: all-model comparison plots
plot_ljung_box_p_values <- function(diagnostics_table) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  ordered_table <-
  diagnostics_table[order(diagnostics_table$Ljung_Box_p_value), ]

  par(mfrow = c(1, 1))
  par(mar = c(5, 16, 4, 2))

  barplot(
    ordered_table$Ljung_Box_p_value,
    names.arg = ordered_table$Model,
    horiz = TRUE,
    las = 1,
    xlab = "Ljung-Box p-value",
    main = "Residual Diagnostics: Ljung-Box p-values"
  )

  abline(v = 0.05, lty = 2)
}

plot_ljung_box_statistics <- function(diagnostics_table) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  ordered_table <-
  diagnostics_table[order(diagnostics_table$Ljung_Box_Statistic), ]

  par(mfrow = c(1, 1))
  par(mar = c(5, 16, 4, 2))

  barplot(
    ordered_table$Ljung_Box_Statistic,
    names.arg = ordered_table$Model,
    horiz = TRUE,
    las = 1,
    xlab = "Ljung-Box statistic",
    main = "Residual Diagnostics: Ljung-Box Statistics"
  )
}

plot_max_abs_residual_acf <- function(diagnostics_table) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  ordered_table <-
  diagnostics_table[order(diagnostics_table$Max_Abs_ACF_Lag_1_to_24), ]

  par(mfrow = c(1, 1))
  par(mar = c(5, 16, 4, 2))

  barplot(
    ordered_table$Max_Abs_ACF_Lag_1_to_24,
    names.arg = ordered_table$Model,
    horiz = TRUE,
    las = 1,
    xlab = "Maximum absolute residual ACF",
    main = "Residual Diagnostics: Maximum Absolute Residual ACF"
  )
}

# Visualization: individual residual diagnostics
plot_residual_time_series <- function(fit, model_name) {
  residuals <- get_clean_residuals(fit)

  plot(
    residuals,
    type = "l",
    main = paste("Residual Plot:", model_name),
    xlab = "Time Index",
    ylab = "Residuals"
  )

  abline(h = 0, lty = 2)
}

plot_residual_acf <- function(fit, model_name, lag.max = 48) {
  residuals <- get_clean_residuals(fit)

  stats::acf(
    residuals,
    lag.max = lag.max,
    main = paste("ACF of Residuals:", model_name)
  )
}

plot_residual_histogram <- function(fit, model_name) {
  residuals <- get_clean_residuals(fit)

  hist(
    residuals,
    breaks = 20,
    main = paste("Histogram:", model_name),
    xlab = "Residuals"
  )
}

plot_residual_qq <- function(fit, model_name) {
  residuals <- get_clean_residuals(fit)

  qqnorm(
    residuals,
    main = paste("Q-Q Plot:", model_name)
  )

  qqline(residuals)
}

plot_full_residual_diagnostics <- function(fit, model_name) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  par(mfrow = c(2, 2))
  par(mar = c(4, 4, 3, 1))

  plot_residual_time_series(fit, model_name)
  plot_residual_acf(fit, model_name)
  plot_residual_histogram(fit, model_name)
  plot_residual_qq(fit, model_name)
}

plot_all_model_diagnostics <- function(models) {
  for (model_name in names(models)) {
    cat("\nPlotting residual diagnostics for:", model_name, "\n")

    plot_full_residual_diagnostics(
      fit = models[[model_name]],
      model_name = model_name
    )

    readline(prompt = "Press [Enter] to continue to the next model...")
  }
}

# Main diagnostics runner
run_model_diagnostics <- function(results, top_n = 4, lag = 24) {
  models <- results$models

  diagnostics_table <- diagnose_all_models(
    models = models,
    lag = lag,
    acf_lag_max = lag
  )

  selected_model_names <- select_models_by_relative_diagnostics(
    diagnostics_table = diagnostics_table,
    top_n = top_n
  )

  diagnostics_table <- add_retention_decision(
    diagnostics_table = diagnostics_table,
    selected_model_names = selected_model_names
  )
}

```

```

cat("\n===== Residual Diagnostics Table =====\n")
print(diagnostics_table, row.names = FALSE, digits = 5)

cat("\n===== Models Retained for Forecasting =====\n")
print(selected_model_names)

cat("\nAll models are ranked by smaller Ljung-Box statistic first, then
smaller maximum absolute residual ACF.\n")

# Visualize all-model comparison before selecting models.
plot_ljung_box_p_values(diagnostics_table)
readline(prompt = "Press [Enter] to continue to Ljung-Box statistic plot...")

plot_ljung_box_statistics(diagnostics_table)
readline(prompt = "Press [Enter] to continue to Max ACF plot...")

plot_max_abs_residual_acf(diagnostics_table)
readline(prompt = "Press [Enter] to continue to individual residual
diagnostics for all models...")

# Visualize every candidate model.
plot_all_model_diagnostics(models)

list(
  diagnostics_table = diagnostics_table,
  selected_model_names = selected_model_names
)
}

# ----- Main -----
set.seed(123)
results <- run_model_estimation(
  file_path = "C:/Users/user/Desktop/NCCU_STAT/Time
Series/final/data/processed/monthly_housing_train.csv"
)

diagnostics_results <- run_model_diagnostics(
  results = results,
  top_n = 4,
  lag = 24
)

```

A.7 Forecasting (R)

```
# =====
# 9. Forecasting
# =====

# -----
# Function
# -----

load_test_data <- function(file_path) {
  df_test <- read.csv(file_path, stringsAsFactors = FALSE)

  required_cols <- c("date", "hpi")
  missing_cols <- setdiff(required_cols, names(df_test))

  if (length(missing_cols) > 0) {
    stop(paste("Missing required columns:", paste(missing_cols, collapse = ",
")))
  }

  df_test$date <- as.Date(df_test$date)
  df_test <- df_test[order(df_test$date), ]

  if (!"log_hpi" %in% names(df_test)) {
    df_test$log_hpi <- log(df_test$hpi)
  }

  df_test <- add_dummy_variables(df_test)

  return(df_test)
}

get_forecast_horizon <- function(df_test) {
  h <- nrow(df_test)
  return(h)
}

make_test_xreg <- function(df_test, model_name) {
  if (grepl("post-2002", model_name)) {
    xreg_test <- as.matrix(df_test[, c("post2002", "crisis"), drop = FALSE])
  } else if (grepl("crisis dummy", model_name)) {
    xreg_test <- as.matrix(df_test[, c("crisis"), drop = FALSE])
  } else {
    xreg_test <- NULL
  }

  return(xreg_test)
}

is_dummy_model <- function(model_name) {
  grepl("^ARIMAX", model_name) || grepl("^SARIMAX", model_name)
}

forecast_level_model <- function(fit, h) {
  fc <- forecast::forecast(

    fit,
    h = h,
    level = c(80, 95)
  )

  forecast_df <- data.frame(
    Forecast_log = as.numeric(fc$mean),
    Lower_80_log = as.numeric(fc$lower[, "80%"]),
    Upper_80_log = as.numeric(fc$upper[, "80%"]),
    Lower_95_log = as.numeric(fc$lower[, "95%"]),
    Upper_95_log = as.numeric(fc$upper[, "95%"])
  )

  forecast_df$Forecast_HPI <- exp(forecast_df$Forecast_log)
  forecast_df$Lower_80_HPI <- exp(forecast_df$Lower_80_log)
  forecast_df$Upper_80_HPI <- exp(forecast_df$Upper_80_log)
  forecast_df$Lower_95_HPI <- exp(forecast_df$Lower_95_log)
  forecast_df$Upper_95_HPI <- exp(forecast_df$Upper_95_log)

  return(forecast_df)
}

forecast_dummy_model <- function(fit, h, xreg_test, last_train_log_hpi) {
  fc <- forecast::forecast(
    fit,
    h = h,
    xreg = xreg_test,
    level = c(80, 95)
  )

  # The dummy models forecast diff(log(HPI)).
  # Convert forecasted monthly growth rates back to log(HPI)
  # by cumulative summation.
  forecast_diff_log <- as.numeric(fc$mean)
  lower_80_diff_log <- as.numeric(fc$lower[, "80%"])
  upper_80_diff_log <- as.numeric(fc$upper[, "80%"])
  lower_95_diff_log <- as.numeric(fc$lower[, "95%"])
  upper_95_diff_log <- as.numeric(fc$upper[, "95%"])

  forecast_log <- last_train_log_hpi + cumsum(forecast_diff_log)
  lower_80_log <- last_train_log_hpi + cumsum(lower_80_diff_log)
  upper_80_log <- last_train_log_hpi + cumsum(upper_80_diff_log)
  lower_95_log <- last_train_log_hpi + cumsum(lower_95_diff_log)
  upper_95_log <- last_train_log_hpi + cumsum(upper_95_diff_log)

  forecast_df <- data.frame(
    Forecast_log = forecast_log,
    Lower_80_log = lower_80_log,
    Upper_80_log = upper_80_log,
    Lower_95_log = lower_95_log,
    Upper_95_log = upper_95_log
  )

  forecast_df$Forecast_HPI <- exp(forecast_df$Forecast_log)
  forecast_df$Lower_80_HPI <- exp(forecast_df$Lower_80_log)
  forecast_df$Upper_80_HPI <- exp(forecast_df$Upper_80_log)
  forecast_df$Lower_95_HPI <- exp(forecast_df$Lower_95_log)
  forecast_df$Upper_95_HPI <- exp(forecast_df$Upper_95_log)

  return(forecast_df)
}

calculate_forecast_accuracy <- function(actual, forecast) {
  error <- actual - forecast

  rmse <- sqrt(mean(error^2))
  mae <- mean(abs(error))
  mape <- mean(abs(error / actual)) * 100

  data.frame(
    RMSE = rmse,
    MAE = mae,
    MAPE = mape,
    row.names = NULL
  )
}

forecast_single_model <- function(model_name, fit, df_test, last_train_log_hpi) {
  h <- get_forecast_horizon(df_test)

  if (is_dummy_model(model_name)) {
    xreg_test <- make_test_xreg(df_test, model_name)

    forecast_df <- forecast_dummy_model(
      fit = fit,
      h = h,
      xreg_test = xreg_test,
      last_train_log_hpi = last_train_log_hpi
    )
  } else {
    forecast_df <- forecast_level_model(
      fit = fit,
      h = h
    )
  }

  forecast_df$date <- df_test$date
  forecast_df$Actual_HPI <- df_test$hpi
  forecast_df$Model <- model_name

  accuracy <- calculate_forecast_accuracy(
    actual = forecast_df$Actual_HPI,
    forecast = forecast_df$Forecast_HPI
  )

  accuracy$Model <- model_name
  accuracy <- accuracy[, c("Model", "RMSE", "MAE", "MAPE")]

  output <- list(

    forecast_df = forecast_df,
    accuracy = accuracy
  )

  return(output)
}

forecast_selected_models <- function(results, diagnostics_results,
test_file_path) {
  df_test <- load_test_data(test_file_path)

  selected_model_names <- diagnostics_results$selected_model_names
  models <- results$models

  last_train_log_hpi <- tail(results$data$log_hpi, 1)

  forecast_outputs <- lapply(selected_model_names, function(model_name) {
    forecast_single_model(
      model_name = model_name,
      fit = models[[model_name]],
      df_test = df_test,
      last_train_log_hpi = last_train_log_hpi
    )
  })

  names(forecast_outputs) <- selected_model_names

  forecast_table <- do.call(
    rbind,
    lapply(forecast_outputs, function(x) x$forecast_df)
  )

  accuracy_table <- do.call(
    rbind,
    lapply(forecast_outputs, function(x) x$accuracy)
  )

  accuracy_table <- accuracy_table[order(accuracy_table$RMSE), ]

  cat("\n===== Forecast Accuracy Table =====\n")
  print(accuracy_table, row.names = FALSE, digits = 5)

  output <- list(
    df_test = df_test,
    selected_model_names = selected_model_names,
    forecast_outputs = forecast_outputs,
    forecast_table = forecast_table,
    accuracy_table = accuracy_table
  )

  return(output)
}

# -----
# Visualization helpers
```

```

# -----
add_english_month_axis <- function(dates, by = 2, show_year = FALSE) {
  tick_index <- seq(1, length(dates), by = by)

  if (show_year) {
    tick_labels <- format(dates[tick_index], "%b %Y")
  } else {
    tick_labels <- format(dates[tick_index], "%b")
  }

  axis(
    side = 1,
    at = dates[tick_index],
    labels = tick_labels
  )
}

plot_forecast_with_intervals <- function(forecast_df, model_name) {
  plot(
    forecast_df$Date,
    forecast_df$Actual_HPI,
    type = "l",
    lwd = 2,
    xaxt = "n",
    ylim = range(
      forecast_df$Lower_95_HPI,
      forecast_df$Upper_95_HPI,
      forecast_df$Actual_HPI,
      na.rm = TRUE
    ),
    xlab = "Date",
    ylab = "HPI",
    main = paste("12-Month HPI Forecast:", model_name)
  )

  add_english_month_axis(
    dates = forecast_df$Date,
    by = 2,
    show_year = FALSE
  )

  lines(
    forecast_df$Date,
    forecast_df$Forecast_HPI,
    lwd = 2,
    lty = 2
  )

  lines(
    forecast_df$Date,
    forecast_df$Lower_95_HPI,
    lty = 3
  )

  lines(
    forecast_df$Date,
    forecast_df$Upper_95_HPI,
    lty = 3
  )

  lines(
    forecast_df$Date,
    forecast_df$Lower_80_HPI,
    lty = 4
  )

  lines(
    forecast_df$Date,
    forecast_df$Upper_80_HPI,
    lty = 4
  )

  legend(
    "topleft",
    legend = c("Actual HPI", "Forecast HPI", "95% PI", "80% PI"),
    lty = c(1, 2, 3, 4),
    lwd = c(2, 2, 1, 1),
    bty = "n"
  )
}

plot_all_selected_forecasts <- function(forecasting_results) {
  selected_model_names <- forecasting_results$selected_model_names

  for (model_name in selected_model_names) {
    cat("\nPlotting forecast for:", model_name, "\n")

    forecast_df <-
      forecasting_results$forecast_outputs[[model_name]]$forecast_df

    plot_forecast_with_intervals(
      forecast_df = forecast_df,
      model_name = model_name
    )

    readline(prompt = "Press [Enter] to continue to the next forecast plot...")
  }
}

plot_forecast_comparison <- function(forecasting_results) {
  forecast_table <- forecasting_results$forecast_table
  df_test <- forecasting_results$df_test
  selected_model_names <- forecasting_results$selected_model_names

  plot(
    df_test$date,
    df_test$hpi,
    type = "l",
    lwd = 2,
    xaxt = "n",
    ylim = range(
      forecast_table$Forecast_HPI,
      df_test$hpi,
      na.rm = TRUE
    ),
    xlab = "Date",
    ylab = "HPI",
    main = "Forecast Comparison for Retained Models"
  )

  add_english_month_axis(
    dates = df_test$date,
    by = 2,
    show_year = FALSE
  )

  for (model_name in selected_model_names) {
    model_forecast <- forecast_table[forecast_table$Model == model_name, ]

    lines(
      model_forecast$Date,
      model_forecast$Forecast_HPI,
      lty = 2
    )

    legend(
      "topleft",
      legend = c("Actual HPI", selected_model_names),
      lty = c(1, rep(2, length(selected_model_names))),
      lwd = c(2, rep(1, length(selected_model_names))),
      bty = "n",
      cex = 0.7
    )
  }

  run_forecasting_analysis <- function(results, diagnostics_results,
    test_file_path) {
    forecasting_results <- forecast_selected_models(
      results = results,
      diagnostics_results = diagnostics_results,
      test_file_path = test_file_path
    )

    plot_forecast_comparison(forecasting_results)
    readline(prompt = "Press [Enter] to continue to individual forecast
      plots...")

    plot_all_selected_forecasts(forecasting_results)

    return(forecasting_results)
  }

  # =====
  # Main
  # =====

  forecasting_results <- run_forecasting_analysis(
    results = results,
    diagnostics_results = diagnostics_results,
    test_file_path = "C:/Users/user/Desktop/NCCU_STAT/Time
      Series/final/data/processed/monthly_housing_test.csv"
  )
}

```